complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() validation logic

filter() null checks + allmatch() validation logic

filter() comparisons + allmatch() validation logic

filter() method calls returning boolean + allmatch() validation logic

distinct() rely on equals() / hashcode() + allmatch() validation logic

distinct() immutable objects + allmatch() validation logic

map() transform objects + allmatch() validation logic

map() create new objects + allmatch() validation logic

map() call pure functions + allmatch() validation logic

flatmap() return stream + allmatch() validation logic

flatmap() split data + allmatch() validation logic

flatmap() expand collections + allmatch() validation logic

sorted() natural ordering + allmatch() validation logic

sorted() custom comparator + allmatch() validation logic

sorted() multi-field compare + allmatch() validation logic

peek() logging + allmatch() validation logic

peek() debugging + allmatch() validation logic

peek() metrics + allmatch() validation logic

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **validation logic** (returns true only if **all elements** satisfy the predicate)
- Demonstrates **short-circuiting** (stops on first failure)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + allMatch() (Validation)

```
import java.util.*;

public class FilterBooleanAllMatch {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean valid = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

2 filter() null checks + allMatch() (Validation)

```
import java.util.*;

public class FilterNullAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean valid = true;

      for (String s : list) {
         if (s != null) {                                    // filter non-null
            if (s.length() == 0) {                           // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

3 filter() comparisons + allMatch() (Validation)

```
import java.util.*;

public class FilterComparisonAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(30, 40, 50);

      boolean valid = true;

      for (Integer age : list) {
         if (age >= 18) {                                    // filter
            if (age > 60) {                                  // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
```

```
    }
}
```

---

4 filter() method calls returning boolean + allMatch() (Validation)

```
import java.util.*;

public class FilterMethodAllMatch {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {

        List<User> list = Arrays.asList(
                new User(true),
                new User(true)
        );

        boolean valid = true;

        for (User u : list) {
            if (u.isActive()) {                                  // filter
                if (!u.isActive()) {                             // allMatch
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + allMatch() (Validation)

```
import java.util.*;

public class DistinctEqualsAllMatch {

   static class Person {
      String name;
      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> list = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> distinct = new HashSet<Person>(list);

      boolean valid = true;

      for (Person p : distinct) {
         if (p.name.length() < 4) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

6 distinct() immutable objects + allMatch() (Validation)

```
import java.util.*;

public class DistinctImmutableAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java", "Java", "Go");

      Set<String> distinct = new HashSet<String>(list);

      boolean valid = true;

      for (String s : distinct) {
         if (s.length() < 2) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

7 map() transform objects + allMatch() (Validation)

```
import java.util.*;

public class MapTransformAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean valid = true;

      for (String s : list) {
         String upper = s.toUpperCase();                    // map

         if (!upper.equals(upper.toUpperCase())) {          // allMatch
            valid = false;
            break;
         }
      }

      System.out.println(valid);
```

```
    }
}
```

---

8 map() create new objects + allMatch() (Validation)

```
import java.util.*;

public class MapCreateObjectAllMatch {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean valid = true;

        for (String s : list) {
            UserDTO dto = new UserDTO(s);                    // map

            if (dto.name.length() < 4) {                     // allMatch
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

---

9 map() call pure functions + allMatch() (Validation)

```
import java.util.*;

public class MapPureFunctionAllMatch {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);

      boolean valid = true;

      for (Integer n : list) {
         int squared = square(n);                    // map

         if (squared <= 0) {                         // allMatch
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

10 flatMap() return Stream + allMatch() (Validation)

```
import java.util.*;

public class FlatMapStreamAllMatch {
   public static void main(String[] args) {

      List<List<Integer>> list = new ArrayList<List<Integer>>();
      list.add(Arrays.asList(1, 2));
      list.add(Arrays.asList(3, 4));

      boolean valid = true;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {                    // flatMap
            if (n <= 0) {                             // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

11 flatMap() split data + allMatch() (Validation)

```
import java.util.*;

public class FlatMapSplitAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");

      boolean valid = true;

      for (String s : list) {
         String[] words = s.split(" ");                    // flatMap

         for (int i = 0; i < words.length; i++) {
            if (words[i].length() < 3) {
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

12 flatMap() expand collections + allMatch() (Validation)

```
import java.util.*;

public class FlatMapExpandAllMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean valid = true;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.length() == 0) {
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

13 sorted() natural ordering + allMatch() (Validation)

```
import java.util.*;

public class SortedNaturalAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      Collections.sort(list);                                    // sorted

      boolean valid = true;

      for (Integer n : list) {
         if (n < 1) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

14 sorted() custom comparator + allMatch() (Validation)

```
import java.util.*;

public class SortedCustomAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Ravi", "Amit");

      Collections.sort(list, Collections.reverseOrder());

      boolean valid = true;

      for (String name : list) {
         if (name.length() < 4) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
```

```
    }
}
```

---

15 sorted() multi-field compare + allMatch() (Validation)

```
import java.util.*;

public class SortedMultiFieldAllMatch {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> list = Arrays.asList(
                new Employee("Anurag", 30),
                new Employee("Anurag", 25),
                new Employee("Ravi", 35)
        );

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0)
                    return nameCompare;
                return e1.age - e2.age;
            }
        });

        boolean valid = true;

        for (Employee e : list) {
            if (e.age < 18) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

---

16 peek() logging + allMatch() (Validation)

```
import java.util.*;

public class PeekLoggingAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean valid = true;

      for (Integer n : list) {
         System.out.println("Checking: " + n);                    // peek

         if (n <= 0) {
            valid = false;
            break;
         }
      }
      System.out.println(valid);
   }
}
```

---

17 peek() debugging + allMatch() (Validation)

```
import java.util.*;

public class PeekDebugAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      boolean valid = true;

      for (Integer n : list) {
         if (n == 20)
            System.out.println("Debug hit");                    // peek

         if (n < 10) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
```

```
    }
}
```

---

18 peek() metrics + allMatch() (Validation)

```
import java.util.*;

public class PeekMetricsAllMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);

        int counter = 0;
        boolean valid = true;

        for (Integer n : list) {
            counter++;                                          // peek

            if (n <= 0) {
                valid = false;
                break;
            }
        }

        System.out.println("Valid: " + valid);
        System.out.println("Processed elements: " + counter);
    }
}
```